

Лабораторная работа №14

Операционные системы

Кац Григорий Михайлович

Российский университет дружбы народов, Москва, Россия

7 октября 2026г.

Информация

Докладчик

- ▶ Кац Григорий Михайлович
- ▶ Студент НКАбд-03-25
- ▶ Российский университет дружбы народов
- ▶ 1032251955@rudn.ru

1. Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2. Задание

1. Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой (`> /dev/tty#`, где `#` — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов.

2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`.
3. Используя встроенную переменную `$RANDOM`, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. Учтите, что `$RANDOM` выдаёт псевдослучайные числа в диапазоне от 0 до 32767.

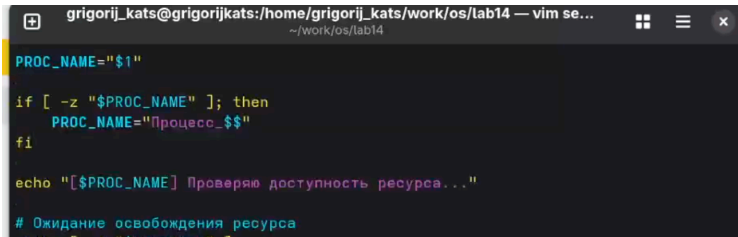
3. Теоретическое введение

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек: – оболочка Борна (Bourne shell или sh) — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций; – C-оболочка (или csh) — надстройка на оболочкой Борна, использующая C-подобный синтаксис команд с возможностью сохранения истории выполнения команд; – оболочка Корна (или ksh) — напоминает оболочку C, но операторы управления программой совместимы с операторами оболочки Борна; – BASH — сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек C и Корна (разработка компании Free Software Foundation).

POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ. Стандарты POSIX разработаны комитетом IEEE (Institute of Electrical and Electronics Engineers) для обеспечения совместимости различных UNIX/Linux-подобных операционных систем и переносимости прикладных программ на уровне исходного кода. POSIX-совместимые оболочки разработаны на базе оболочки Корна.

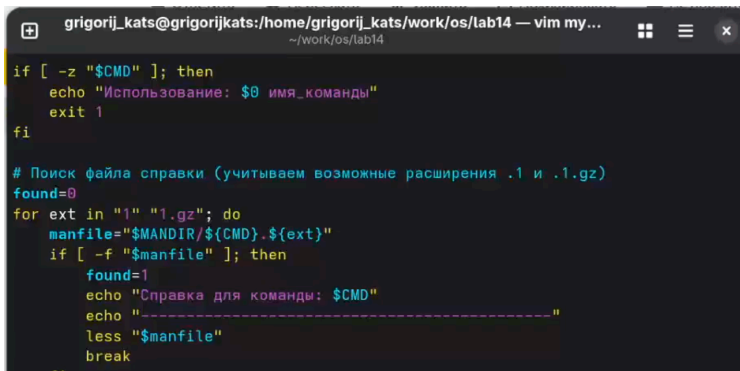
4. Выполнение лабораторной работы

1. Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой, в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов.



```
grigorij_kats@grigorijkats:/home/grigorij_kats/work/os/lab14 — vim se...  
~/work/os/lab14  
PROC_NAME="$1"  
  
if [ -z "$PROC_NAME" ]; then  
    PROC_NAME="Процесс_$$"  
fi  
  
echo "[$PROC_NAME] Проверяю доступность ресурса..."  
  
# Ожидание освобождения ресурса
```


2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`.



```
grigorij_kats@grigorijkats:/home/grigorij_kats/work/os/lab14 — vim my...
~/work/os/lab14

if [ -z "$CMD" ]; then
    echo "Использование: $0 имя_команды"
    exit 1
fi

# Поиск файла справки (учитываем возможные расширения .1 и .1.gz)
found=0
for ext in "1" "1.gz"; do
    manfile="$MANDIR/${CMD}.${ext}"
    if [ -f "$manfile" ]; then
        found=1
        echo "Справка для команды: $CMD"
        echo "-----"
        less "$manfile"
        break
    fi
done
```

3. Используя встроенную переменную \$RANDOM, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. Учтите, что \$RANDOM выдаёт псевдослучайные числа в диапазоне от 0 до 32767.

```
grigorij_kats@grigorijkats:/home/grigorij_kats/work/os/lab14 — vim ran...
~/work/os/lab14

#!/bin/bash
# Генерирует случайную последовательность латинских букв через $RANDOM
# Использование: ./random_letters.sh N (количество букв)

N="${1:-10}"

result=""
i=0
while [ $i -lt $N ]
do
    # $RANDOM: 0..32767, латинский алфавит: 26 букв (A-Z: 65-90 в ASCII)
    rand_val=$((RANDOM % 26))
    ascii_code=$((65 + rand_val))
    letter=$(printf "\\$(printf '%03o' $ascii_code)")
    result="${result}${letter}"
    i=$((i+1))
done

echo "Случайная последовательность из $N букв: $result"
~
~
```

5. Выводы

Мы изучили основы программирования в оболочке ОС UNIX. Научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов